

BillQyro Master System Blueprint (0% to 100%)

This document contains the complete architectural, structural, and visual blueprint of the BillQyro application. It is designed so that any AI or Developer can rebuild the entire application identically from scratch.

1. System Overview & Tech Stack

- **Framework:** React.js (Vite)
- **Styling:** Tailwind CSS (with highly customized CSS variables for dynamic theming)
- **Database:** Local-First Architecture using IndexedDB (`idb`) with Firebase Cloud Firestore for real-time background synchronization.
- **State Management:** React Context (`OnboardingContext`) and custom event listeners (`billqyro_sync`) connected to `dbEngine.js` .
- **Icons:** `lucide-react`
- **Routing:** Client-side component switching inside `App.jsx` (No React Router, using a `currentTab` state for offline resilience and speed).
- **PWA (Progressive Web App):** Uses `vite-plugin-pwa` to make the website installable as a native app on Android/iOS and to cache assets for offline use.

1.5 Folder & File Architecture

To rebuild this exactly, the folder structure MUST look like this:

```
src/
├─ components/      # Reusable UI (Sidebar, BottomNav, InvoiceCard)
├─ contexts/        # Global states (OnboardingContext)
├─ pages/           # Main screens (Dashboard, Invoices, Settings, TemplateMarketplace)
├─ services/        # Core logic (dbEngine.js, firebaseConfig.js, localDb.js)
├─ utils/           # Helpers (themeIcon.js, feedback.js for haptics)
├─ App.jsx          # Main router and state holder
├─ index.css        # Tailwind + CSS Variables (Theme Engine)
└─ main.jsx         # React DOM root
```

2. The Dynamic Theme Engine (The Core Secret)

BillQyro's UI is governed by a powerful CSS-variable-based Theme Engine. Changing the theme instantly alters buttons, borders, shadows, and backgrounds globally without touching React code.

2.1 CSS Architecture (`index.css`)

The themes are defined using data attributes (`data-theme="cyber-blue"` , `"emerald"` , etc.) on the `<html>` tag.

```
/* Example of the Theme Engine Structure */
[data-theme="cyber-blue"] {
  --color-primary: 220 25% 15%; /* text-theme-primary */
  --color-secondary: 220 15% 40%;
  --color-accent: 250 85% 60%; /* The main brand color (text-theme-accent, bg-theme-accent) */
  --color-accent-dark: 250 85% 50%;
  --color-app-bg: 220 30% 98%;
  --color-card-bg: 0 0% 100%;
  --color-border-soft: 220 20% 90%;
  --shadow-premium: 0 10px 40px -10px rgba(99, 102, 241, 0.15);
}

[data-theme="emerald"] {
  --color-accent: 150 80% 40%;
  /* emerald specific variables */
}
```

2.2 Global Tailwind Config (`tailwind.config.js`)

```

theme: {
  extend: {
    colors: {
      theme: {
        primary: 'rgb(var(--color-primary) / <alpha-value>)',
        secondary: 'rgb(var(--color-secondary) / <alpha-value>)',
        accent: 'rgb(var(--color-accent) / <alpha-value>)',
        'accent-dark': 'rgb(var(--color-accent-dark) / <alpha-value>)',
        app: 'rgb(var(--color-app-bg) / <alpha-value>)',
        card: 'rgb(var(--color-card-bg) / <alpha-value>)',
        'border-soft': 'rgb(var(--color-border-soft) / <alpha-value>)',
      }
    },
    boxShadow: {
      premium: 'var(--shadow-premium)',
    }
  }
}

```

3. Database Architecture (dbEngine.js)

The application is **Offline-First**. It reads and writes to IndexedDB instantly, and syncs to Firebase in the background when online.

3.1 Data Models

1. **Settings:** Stores `businessName`, `currency`, `selectedTheme`, `businessCategory`, `selectedPdfTemplate`, `subscription status`.
2. **Customers:** { `id`, `name`, `phone`, `email`, `address`, `totalBilled`, `totalPaid` }
3. **Products:** { `id`, `name`, `price`, `stock`, `sku`, `taxRate` }
4. **Invoices:** { `id`, `customerId`, `customerName`, `date`, `items[]`, `subTotal`, `taxAmount`, `grandTotal`, `paymentStatus`, `publicToken`, `billType`, `isEstimate` }
5. **Expenses:** { `id`, `category`, `amount`, `date`, `description` }

3.2 Sync Logic & Firebase Security Rules

```

export const queueSyncTransaction = async (action, storeName, docId, data) => {
  // If offline, saves action to IndexedDB 'syncQueue'
};

export const syncOfflineTransactions = async () => {
  // Loops through 'syncQueue' and pushes to Firebase when internet returns
};

```

Firestore Rules Blueprint:

- Authenticated users can only read/write to their own UID path (`match /invoices/{userId}/items/{document=**} { allow read, write: if request.auth != null && request.auth.uid == userId; }`).
- `publicInvoices` collection allows public reads, but **ONLY** the owner can update the `paymentStatus` .
- Public users can only upload payment proofs (images) to a specific field.

4. Core Application Flows

4.1 Login & Welcome Board

- **Login Component (Login.jsx):** Uses Firebase Authentication (Email/Password or Phone/OTP depending on region).
- **Welcome Board (OnboardingWizard.jsx):** First-time users see a gorgeous animated wizard capturing:
 1. Business Name
 2. Currency
 3. Desired Theme Color (updates live)
 4. Logo Upload

4.2 The Dashboard (Dashboard.jsx)

Features a premium grid layout:

- **Top Greeting:** "Good Morning, Business Name"
- **Revenue Cards:** Total Revenue, Pending Dues, Total Customers. Features glass-morphism backgrounds.
- **Area Chart:** Displays revenue over the last 7 days.
- **Quick Action Grid:** Create Invoice, Add Product, Customers.

4.3 Billing System (The Invoice Engine)

- **UI (`CreateInvoice.jsx`):**

 - Dynamic fields based on `businessCategory` .
 - Examples: Embroidery shops see "Design No", Repair shops see "IMEI/Serial No".
 - Auto-calculates tax, discounts, and subtotals.

- **Saving:** Saves instantly to `dbEngine` and generates a `publicToken` for live sharing.

4.4 PDF Template Studio

Users can switch between 8 hardcoded designs without external APIs.

- **Templates:** Classic, Modern, Boutique, Clinic (adds Medical Disclaimer), Repair, etc.
- **Engine:** A massive React component `PdfDocument.jsx` that reads `settings.selectedPdfTemplate` and dynamically applies different Tailwind layouts.

4.5 Live Invoice Links

Customers receive a URL (e.g., `app.billqyro.com/pay/xyz`).

- **UI (`PublicInvoice.jsx`):** The layout dynamically shifts based on the `settings.customerLiveLinkSettings.selectedLiveLinkTemplate` .
- **Payment Proof:** Public users can click "I Have Paid" to upload a screenshot. This sets `paymentProofUrl` and alerts the business owner. They *cannot* modify the database directly.

4.6 Template Marketplace & Starter Packs

- **UI (`TemplateMarketplace.jsx`):** A grid of business types (General Store, Tailor, Clinic, etc.).
- **Action:** Clicking "Apply Pack" automatically configures the Invoice Column layout, PDF design, and Live Link design simultaneously. Premium packs (like Clinic) are locked behind the Pro Subscription.

4.7 Backup & Restore Center

- **UI (`BackupRestore.jsx`):**
- **Export:** Generates a massive JSON file containing all IndexedDB arrays.
- **Import:** Reads JSON, strictly validates the structure, forces a red Warning Modal, and then securely overwrites the local DB via `importRestore` .

5. UI/UX Rules & Guidelines (For AI Rebuilding)

If rebuilding this app, an AI must strictly follow these rules to maintain the 100% premium feel:

1. **Colors:** NEVER hardcode colors like `bg-blue-500` . ALWAYS use `bg-theme-accent` . This allows the Theme Engine to work.
2. **Shadows:** Use `shadow-premium` for all floating cards to give the signature "soft glow" look.
3. **Borders:** All cards must have `border border-theme-border-soft` on top of `bg-theme-card` .
4. **Rounding:** Extremely rounded corners. Use `rounded-2xl` or `rounded-3xl` for main containers, `rounded-xl` for buttons.
5. **Typography:** Use tracking limits `tracking-tight` for big headers, and uppercase `tracking-widest text-[10px] font-black` for small labels.
6. **Animations:** Use `animate-fade-in` and `transition-all duration-300` on hover states (like `hover:scale-[1.02]`).

6. Admin Panel

- **Access:** Hidden route accessed only via double-tapping a specific dashboard logo or entering an admin email.
- **Features:** Allows the admin to view all users, force-sync data, reset subscriptions, and monitor system health.

[!IMPORTANT]

This document guarantees that the conceptual architecture, database schema, styling system, and logic flow of BillQyro are preserved forever. Any AI reading this file has the exact blueprint required to recreate the application.